



OXETECH Academy



SECTI
Secretaria de Estado de
Tecnologia e Inovação



 Softex

REGISTERED BY
AGÊNCIA REGULADORA
DE PROTEÇÃO

CONSUMIDOR FEDERAL
BRASIL
CONHEÇA O SEU DIREITO DE PROTEÇÃO

Cronograma



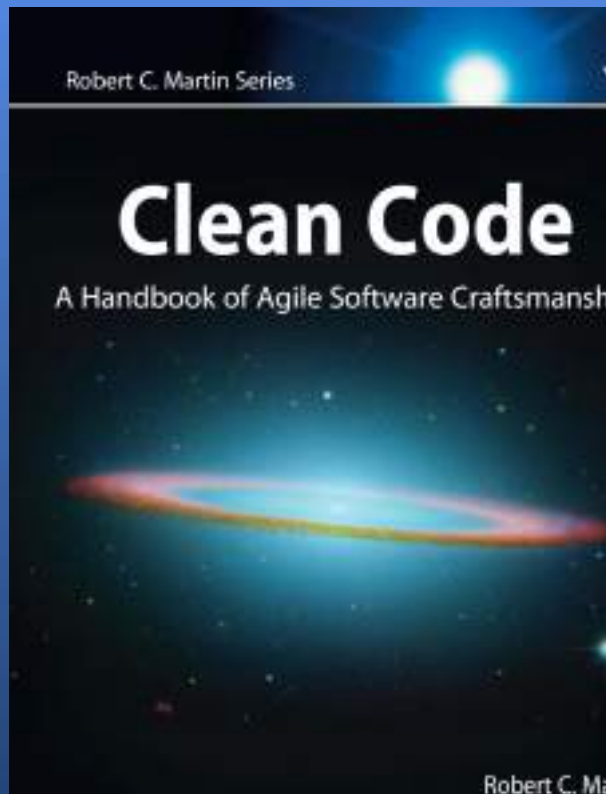
Módulos	Conteúdo
Módulo 1 — Fundamentos da Engenharia Moderna	Processos, versionamento e code review Gestão de dependências, Documentação viva
Módulo 2 — Clean Code e SOLID	Legibilidade, funções e acoplamento, SOLID aplicado, Refatoração incremental
Módulo 3 — Back -end com Node.js	Factory, Strategy, Observer, Repository, Adapter, Decorator, Anti -patterns comuns

Módulos	Conteúdo
Módulo 4 — Arquiteturas e Escalabilidade	Monólito modular, hexagonal, Microserviços e mensageria, DDD básico
Módulo 5 — Qualidade, Testes e Segurança	Pirâmide de testes e cobertura, Sonar, linters, quality gates, Segurança básica em apps
Módulo 6 — CI/ CD e Operação	Pipelines (build/ test/ deploy), Blue green, canary e rollback, Observabilidade (logs/ metrics/ traces)
Módulo 7 — Projeto Final Integrador	Sistema modular com integração contínua, Qualidade e observabilidade integradas, Defesa técnica

Revisão



Módulo 2 — Clean Code e SOLID



O livro *Clean Code* de Robert C. Martin (Uncle Bob) defende que **programar não é apenas fazer o código funcionar, mas fazê-lo funcionar de forma clara, legível e sustentável**. O código limpo é aquele que pode ser facilmente entendido, mantido e evoluído por qualquer desenvolvedor, hoje ou no futuro.

O que é Clean Code?



Clean Code é um conjunto de princípios e práticas que tornam o código mais legível, fácil de entender, manter e evoluir. Ele não se preocupa apenas em “funcionar”, mas em ser sustentável e claro para qualquer desenvolvedor que o leia.

Por que escrever código limpo é importante



Escrever código limpo é importante porque garante que o software seja mais fácil de entender, manter e evoluir, reduzindo custos e riscos a longo prazo. Ele aumenta a produtividade da equipe, facilita a colaboração e evita que o sistema se torne frágil ou difícil de escalar.

Diferença entre “código que funciona” e “código sustentável”



Código que Funciona	Código Sustentável (Clean Code)
Resolve o problema imediato.	Vai além de funcionar: é legível, modular e testável .
Foco em entregar rápido, sem preocupação com clareza.	Segue princípios como simplicidade, baixo acoplamento e alta coesão .
Pode conter duplicações, nomes confusos e lógica complexa	Facilita manutenção, refatoração e evolução contínua.
Funciona hoje, mas dificulta manutenção e evolução .	Permite que diferentes desenvolvedores entendam e modifiquem sem riscos.

Princípios Fundamentais

Legibilidade	• Código deve ser fácil de entender por qualquer desenvolvedor. • Nomes claros e significativos. • Estrutura organizada e consistente.
Simplicidade	• Evitar complexidade desnecessária. • Soluções simples são mais robustas e fáceis de manter.
Funções Pequenas e Focadas	• Cada função deve fazer apenas uma coisa. • Poucos parâmetros. • Modularidade facilita testes e manutenção.

Responsabilidade Única (SRP)	• Cada classe ou módulo deve ter apenas uma razão para mudar. • Promove coesão e clareza.
Baixo Acoplamento e Alta Coesão	• Módulos independentes entre si. • Cada parte do sistema focada em sua responsabilidade. • Facilita evolução e manutenção.
Testabilidade	• Código limpo é acompanhado de testes automatizados. • Testes devem ser legíveis, rápidos e confiáveis.
Refatoração Contínua	• Aplicar a “Regra do Escoteiro”: sempre deixar o código melhor do que encontrou. • Melhorias incrementais garantem longevidade do software.

Legibilidade, funções e acoplamento

Principais Práticas para Legibilidade (Clean Code):

- **Nomes Significativos:** Variáveis, funções e classes devem revelar claramente sua intenção. Evite nomes genéricos como **data** ou **temp**.
- **Funções Pequenas e Únicas:** Funções devem fazer apenas uma coisa e fazê-la bem, com poucos parâmetros.
- **Evitar Comentários Desnecessários:** O código deve ser autoexplicativo. Comentários devem ser usados apenas para explicar o "porquê", não o "quê".
- **Código Simples e Direto:** Evite lógica complexa e desnecessária. O código deve ser organizado e consistente.
- **Remover Código Morto:** Código não utilizado ou comentado deve ser removido para evitar confusão

Principais Práticas para Legibilidade (Clean Code):

- **Legibilidade:** Código claro é mais fácil de ler e entender.
- **Manutenção:** Funções menores são mais fáceis de modificar sem quebrar outras partes do sistema.
- **Reutilização:** Funções focadas em uma única tarefa (ex: `calcular_media(notas)`) podem ser reutilizadas em diferentes partes do software.

Dicas para manter o foco:

- **Regra de Ouro:** Se uma função está muito grande, considere dividi-la em subfunções.
- **Responsabilidade Única:** Cada função deve realizar apenas uma ação e fazê-la bem.
- **Nomes Claros:** Dê nomes que explicam o propósito da função, facilitando o entendimento do código sem precisar ler a implementação interna. [

Boas Práticas



Regra do Escoteiro	• Sempre deixe o código melhor do que encontrou. • Refatorar continuamente, mesmo em pequenas melhorias.
Organização Vertical e Proximidade Lógica	• Agrupar código relacionado. • Declarar variáveis próximas ao uso. • Manter funções relacionadas próximas umas das outras.
Encapsulamento e Coesão em Classes	• Esconder detalhes internos, expondo apenas o necessário. • Cada classe deve ter uma responsabilidade clara e única. • Alta coesão: manter foco em uma única tarefa.

Benefícios do Código Limpo



Redução da Dívida Técnica	• Menos retrabalho e menos acúmulo de problemas futuros. • Evita que o sistema se torne frágil e difícil de evoluir.
Facilidade de Manutenção	• Qualquer desenvolvedor consegue entender e modificar o código com segurança. • Menos tempo gasto decifrando trechos confusos.
Maior Produtividade da Equipe	• Equipes trabalham mais rápido e com menos erros. • Novos membros conseguem se integrar facilmente ao projeto.

Exemplos de Casos de Sucesso e Código



Spotify

- Ao aplicar princípios de Clean Code e refatoração contínua, conseguiram escalar sua plataforma para milhões de usuários sem perder agilidade.
- Resultado: ciclos de entrega mais rápidos e menos bugs críticos.

Netflix

- Investiu em práticas de código limpo e testes automatizados para suportar crescimento global.
- Resultado: sistema resiliente, capaz de evoluir rapidamente com novas funcionalidades.

Etsy

- Reescreveu partes críticas do sistema aplicando Clean Code e padrões de projeto.
- Resultado: maior estabilidade e facilidade de integração de novos desenvolvedores.

Clean Code: Nomes Significativos



Regras para usar nomes claros, pronunciáveis e pesquisáveis

Regra 1: Use Nomes Claros e Descritivos

✗ Ruim

```
const d = new Date();
const yyymmdstr = d.getFullYear();
let list = [];
```

Nomes genéricos não explicam a intenção

✓ Bom

```
const currentDate = new Date();
const formattedDate = currentDate.getFullYear();
const numbers = [];
```

Nomes descritivos deixam claro o propósito

Regra 2: Use Nomes Pronunciáveis

✗ Ruim

```
class DtaRcrd102 {
  genymdhms() { }
  pszqint = '102';
}
```

Abreviações crípticas são impossíveis de pronunciar

✓ Bom

```
class Customer {
  generationTimestamp() { }
  recordId = '102';
}
```

Nomes pronunciáveis são fáceis de lembrar

Regra 3: Use Nomes Pesquisáveis

✗ Ruim

```
for (let j = 0; j < 34; j++) {
  s += (t[j] * 4) / 5;
}
```

Números mágicos são impossíveis de pesquisar

✓ Bom

```
const WORK_DAYS = 5;
for (let day = 0; day < totalDays; day++) {
  effort += (tasks[day] * 4) / WORK_DAYS;
}
```

Nomes descritivos são fáceis de pesquisar

Clean Code: Funções



Regras para escrever funções pequenas que fazem uma coisa bem

Regra 1: Funções Devem Fazer Uma Coisa

✗ Ruim: Múltiplas Responsabilidades

```
function validateAndSaveUser(data) {  
  // Validação  
  if (!data.email.includes('@')) {  
    throw new Error('Email inválido');  
  }  
  // Transformação  
  const user = { email: data.email.toLowerCase() };  
  // Salvamento  
  database.save(user);  
  emailService.send(user.email);  
}
```

✓ Bom: Uma Responsabilidade

```
function validateUser(data) {  
  if (!data.email.includes('@')) {  
    throw new Error('Email inválido');  
  }  
}  
  
function registerUser(data) {  
  validateUser(data);  
  const user = normalizeUser(data);  
  database.save(user);  
  emailService.send(user.email);  
}
```

Regra 2: Funções Devem Ser Pequenas

Ideal: 1-5 linhas | Aceitável: até 20 linhas | Se > 20 linhas, provavelmente faz mais de uma coisa

Regra 3: Evite Muitos Parâmetros

✗ Ruim: Muitos Parâmetros

```
function createUser(name, email, age, phone,  
  address, city, country, zipCode) {  
  // ...  
}  
  
createUser('João', 'joao@email.com',  
  30, '123456789', 'Rua A',  
  'São Paulo', 'Brasil', '01310')
```

Difícil de lembrar ordem dos parâmetros

✓ Bom: Objeto como Parâmetro

```
function createUser(userData) {  
  const { name, email, age, phone,  
    address, city, country } = userData;  
  // ...  
}  
  
createUser({  
  name: 'João', email: 'joao@email.com',  
  age: 30, phone: '123456789',  
  address: 'Rua A', city: 'São Paulo'  
})
```

Clean Code: Comentários e Documentação



Quando usar comentários e quando deixar o código falar por si

✓ Bom: Quando Usar Comentários

```
// Explicar POR QUE, não o quê
// Precisamos fazer cache porque
// a API externa é lenta (100ms+)
const cachedResults = new Map();

// ATENÇÃO: Este método modifica
// o array original
function sortInPlace(array) { }

// Usamos SHA256 em vez de MD5
// por segurança (MD5 é quebrado)
const hash = crypto.createHash('sha256');
```

✗ Ruim: Quando Evitar Comentários

```
// Incrementar i
i++;

// Verificar se email é válido
if (email.includes('@')) { }

// Criar novo usuário
const user = new User(name, email);

// Retornar o resultado
return result;
```

Regra Principal: Código Limpo Não Precisa de Comentários

✗ Ruim: Código Complexo + Comentário

```
// Verificar se o usuário é admin
// e tem permissão para deletar
if (user.role === 'admin' &&
    user.permissions.includes('delete')) {
  deleteUser(userId);
}
```

Comentário tenta explicar lógica complexa.

✓ Bom: Código Limpo = Sem Comentário

```
if (userCanDeleteOtherUsers(user)) {
  deleteUser(userId);
}

function userCanDeleteOtherUsers(u) {
  return u.role === 'admin' &&
    u.permissions.includes('delete');
}
```

Código se explica através de nomes claros.

Clean Code: Tratamento de Erros



Usar exceções, não ignorar erros, tratamento adequado

Regra 1: Use Exceções, Não Códigos de Erro

✗ Ruim: Retornar Código de Erro

```
function getUser(id) {
  const user = db.findById(id);
  if (!user) {
    return {
      error: 'USER_NOT_FOUND',
      code: 404;
    };
  }
  return { success: true, data: user };
}

const result = getUser(123);
if (result.error) { console.log(result.error); }
```

✓ Bom: Usar Exceções

```
function getUser(id) {
  const user = db.findById(id);
  if (!user) {
    throw new UserNotFoundError(
      'Usuário ${id} não encontrado'
    );
  }
  return user;
}

try {
  const user = getUser(123);
} catch (error) {
  if (error instanceof UserNotFoundError) {
    console.log(error.message);
  }
}
```

Regra 2: Não Ignore Exceções

✗ Ruim: Silenciosamente Ignorar Erro

```
try {
  saveUser(user);
} catch (error) {
  // Silenciosamente ignora erro
  // Problema: nunca saberemos
  // que falhou!
}
```

Erro silencioso causa bugs difíceis de rastrear.

✓ Bom: Log e Re-lançar

```
try {
  saveUser(user);
} catch (error) {
  logger.error(
    'Erro ao salvar usuário',
    error
  );
  throw error; // Ou tratar se recuperável:
  // notifyUser('Falha ao salvar');
}
```

Vamos praticar?



SOLID aplicado

S.O.L.I.D.



Definição: SOLID é um conjunto de **cinco princípios de design de software orientado a objetos**, criados para tornar o código mais **legível, flexível e fácil de manter**.

Origem: Popularizados por Robert C. Martin (Uncle Bob), são considerados pilares da programação limpa e da engenharia de software moderna.

Objetivo:

- Reduzir acoplamento entre módulos
- Aumentar coesão e clareza
- Facilitar testes e evolução do sistema
- Promover boas práticas de arquitetura

Os Cinco Princípios



S – Single Responsibility Principle (SRP)	Cada classe deve ter apenas uma responsabilidade.
O – Open/Closed Principle (OCP)	Aberto para extensão, fechado para modificação
L – Liskov Substitution Principle (LSP)	Subclasses devem poder substituir superclasses sem problemas.
I – Interface Segregation Principle (ISP)	Interfaces devem ser pequenas e específicas.
D – Dependency Inversion Principle (DIP)	Depender de abstrações, não de implementações concretas.

Boas Práticas do SOLID



Aplicar SRP (Single Responsibility Principle)	• Mantenha cada classe focada em uma única responsabilidade. • Evite classes “faz-tudo”. • Refatore sempre que perceber múltiplas razões para mudança.
Seguir OCP (Open/Closed Principle)	• Estruture o código para ser extensível sem precisar modificar o que já funciona. • Use herança, composição ou padrões de projeto (ex.: Strategy, Decorator).
Garantir LSP (Liskov Substitution Principle)	• Subclasses devem respeitar o contrato da classe base. • Evite sobrescrever métodos quebrando expectativas. • Teste substituições para garantir consistência.

Praticar ISP (Interface Segregation Principle)	• Crie interfaces pequenas e específicas. • Evite interfaces “inchadas” que obrigam classes a implementar métodos inúteis. • Prefira várias interfaces coesas a uma única interface genérica.
Adotar DIP (Dependency Inversion Principle)	• Dependenda de abstrações, não de implementações concretas. • Use injeção de dependência para reduzir acoplamento. • Facilite testes substituindo dependências por mocks ou stubs.
D – Dependency Inversion Principle (DIP)	• Dependenda de abstrações, não de implementações concretas.

Exemplos com Código



SOLID: Single Responsibility Principle



Cada classe deve ter uma única responsabilidade

O SRP diz que uma classe deve ter apenas uma razão para mudar. Se uma classe tem múltiplas responsabilidades, mudanças em uma podem quebrar outra.

❌ Ruim: Múltiplas Responsabilidades

```
class User {  
  constructor(name, email) {  
    this.name = name;  
    this.email = email;  
  }  
  getName() { return this.name; }  
  validateEmail() { /* ... */ }  
  sendEmail(msg) { /* ... */ }  
  saveToDatabase() { /* ... */ }  
}
```

Classe User tem 4 responsabilidades diferentes. Mudança em validação de email pode quebrar a classe.

✓ Bom: Uma Responsabilidade por Classe

```
class User {  
  constructor(name, email) {  
    this.name = name;  
    this.email = email;  
  }  
  getName() { return this.name; }  
}  
  
class EmailValidator { /* ... */ }  
class EmailService { /* ... */ }  
class UserRepository { /* ... */ }
```

Cada classe tem uma única responsabilidade. Fácil de testar, modificar e reutilizar.

SOLID: Open/Closed, Liskov, Interface, Dependency



Continuação dos princípios SOLID com exemplos práticos

O - Open/Closed Principle: Aberto para Extensão, Fechado para Modificação

❌ Ruim: Modificar Classe Existente

```
class PaymentProcessor {  
  processPayment(type, amt) {  
    if (type === 'credit') {  
      // Lógica cartão  
    } else if (type === 'debit') {  
      // Lógica débito  
    } else if (type === 'paypal') {  
      // Lógica PayPal  
    }  
  }  
}
```

Adicionar novo tipo de pagamento requer modificar a classe existente.

✅ Bom: Strategy Pattern

```
class PaymentProcessor {  
  constructor(strategy) {  
    this.strategy = strategy;  
  }  
  processPayment(amt) {  
    return this.strategy.process(amt);  
  }  
}
```

```
class BitcoinStrategy {  
  process(amt) {  
    // ...  
  }  
}
```

Adicionar novo tipo cria nova classe, sem modificar existente.

L - Liskov Substitution

Subclasses devem ser substituíveis por suas superclasses sem quebrar o código.

I - Interface Segregation

Clientes não devem depender de interfaces que não usam.

D - Dependency Inversion

Dependa de abstrações, não de implementações concretas.

Vamos praticar?



Refatoração incremental



Refatoração é uma prática essencial em Engenharia de Software que melhora a estrutura interna do código sem alterar seu comportamento externo. Ela reduz a dívida técnica, aumenta a clareza e facilita a manutenção contínua.

Práticas Comuns



Remover duplicações	Eliminar trechos repetidos.
Simplificação de lógica condicional:	Tornar estruturas de decisão mais claras.
Extração de métodos e classes:	Dividir funções ou classes grandes em partes menores e coesas.
Renomeação de variáveis e funções	Usar nomes significativos para aumentar legibilidade.
Seguir padrões de codificação	Manter consistência para facilitar refatorações futuras.

Exemplos de Técnicas



Extract Method	Transformar um trecho de código em uma função separada.
Extract Class	Dividir uma classe com muitas responsabilidades em classes menores.
Inline Method	Remover métodos desnecessários incorporando-os diretamente.
Replace Temp with Query	Substituir variáveis temporárias por chamadas de métodos claros.

Casos de Aplicação





Métricas de Qualidade de Código



Ferramentas e métricas para medir e melhorar a qualidade

1. Complexidade Ciclomática

O que é: Número de caminhos diferentes que o código pode executar

Ideal: < 10 por função

Ferramenta: SonarQube, ESLint

Cada if/else, switch case, loop aumenta a complexidade

2. Cobertura de Testes

O que é: Percentual de código coberto por testes automatizados

Ideal: > 80% (crítico > 90%)

Ferramenta: Jest, Codecov, Istanbul

Não é apenas cobertura, mas qualidade dos testes

3. Duplicação de Código

O que é: Percentual de código duplicado no projeto

Ideal: < 5%

Ferramenta: SonarQube, Duplicate Finder

Código duplicado = manutenção duplicada e bugs

4. Índice de Manutenibilidade

O que é: Métrica combinada de complexidade, cobertura e duplicação

Ideal: > 80 (verde)

Ferramenta: SonarQube, Code Climate

Visão holística da qualidade do código

Case Real: Refatoração de E-commerce



Exemplo prático de transformação de código legado para código limpo

Cenário: E-commerce com 5 Anos

Problema Inicial:

- Função processOrder com 200+ linhas
- 15+ responsabilidades diferentes
- Difícil de testar
- Bugs frequentes
- Manutenção cara e lenta

Impacto:

- 2 semanas para adicionar feature
- 30% do tempo em bugs
- Rotatividade alta de devs

Resultado

Código Principal

200 linhas → 20 linhas

Testabilidade

Cada classe testável isoladamente

Manutenção

Mudanças localizadas e seguras

Refatoração Incremental

Passo 1: Extrair Validação

OrderValidator class com lógica isolada

Passo 2: Extrair Cálculo de Preço

PriceCalculator com descontos e taxas

Passo 3: Extrair Processamento de Pagamento

PaymentProcessor com Strategy Pattern

Passo 4: Extrair Notificações

OrderNotificationService isolada

Impacto Financeiro

Tempo de Desenvolvimento

2 semanas → 2 dias (10x mais rápido)

Bugs Reduzidos

30% → 5% (6x menos bugs)

Satisfação da Equipe

Código legível e manutenível

Vamos praticar?





OXETECH Academy



SECTI
Secretaria de Estado de
Tecnologia e Inovação
e Inovação



 Softex

REGISTERED BY
AGÊNCIA REGULADORA
DE PROTEÇÃO

CONSUMIDOR FEDERAL
BRASIL
CONSUMIDOR FEDERAL